Singleton Design Pattern

1. What is Singleton Design Pattern?

Singleton Design Pattern ensures that a class has **only one instance** throughout the entire execution of a program and provides a **global access point** to that instance.

In simple words, Singleton controls object creation so that multiple objects of the same class cannot exist.

2. Problem Addressed by Singleton

In many systems, allowing multiple objects of a class can lead to serious problems such as:

- Inconsistent data
- Resource overuse
- Conflicting system states

Singleton solves this by allowing **exactly one shared object** to coordinate system-wide actions.

3. When to Use Singleton

Singleton is suitable when:

- Exactly one object is required
- The object must be globally accessible
- Centralized control is needed

Typical examples:

- Database connection manager
- Configuration manager
- Logging service
- Cache or resource manager

4. Core Implementation Principles

Any correct Singleton implementation must satisfy:

- **Private constructor** — prevents object creation using `new`
- **Static instance variable** — stores the single object
- **Public access method** — returns the instance

5. Basic Singleton (Single-Threaded)

```
public class Singleton {

    private static Singleton instance;

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    private Singleton() { }
}
```

Limitation: This implementation is **NOT thread-safe**.

6. Issue in Multi-Threaded Environment

If two threads access `getInstance()` at the same time:

- Both may find `instance == null`
- Both create separate objects
- Singleton guarantee is violated

This is known as a **race condition**.

7. Thread-Safe Singleton using Synchronization

```
public class Singleton {

    private static Singleton instance;
```

```
    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    private Singleton() { }
}
```

Pros:

- Thread-safe
- Lazy initialization

Cons:

- Synchronization causes performance overhead

8. Eager Initialization Singleton

```
public class Singleton {

    private static final Singleton INSTANCE = new Singleton();

    public static Singleton getInstance() {
        return INSTANCE;
    }

    private Singleton() { }
}
```

Characteristics:

- Object created at class loading time
- Thread-safe without synchronization
- May waste memory if unused

9. Breaking Singleton using Reflection

Java Reflection can access private constructors and create new objects.

```
private Singleton() {
    if (INSTANCE != null)
        throw new IllegalStateException("Instance␣already␣exists");
}
```

This prevents reflection-based attacks.

10. Enum-Based Singleton (Best Practice)

```
public enum Singleton {
    INSTANCE;
}
```

This is the **most robust Singleton implementation in Java**.

Why Enum Singleton is Best:

- Thread-safe by default
- Safe from reflection attacks
- Handles serialization automatically
- Minimal and clean code

11. When NOT to Use Singleton

- When multiple instances are actually needed
- When unit testing requires object mocking
- When global state causes tight coupling

12. Comparison of Singleton Approaches

Approach	Thread-Safe	Lazy	Secure	Recommended
Basic	No	Yes	No	No
Synchronized	Yes	Yes	No	Rare
Eager	Yes	No	Partial	Sometimes
Enum	Yes	No	Yes	**Yes**

13. Exam Tip

If asked: *"Which Singleton implementation is best in Java?"* Answer: **Enum-based Singleton**, as recommended by Joshua Bloch.

14. Conclusion

Singleton Design Pattern is used to restrict object creation to a single instance. For modern Java applications, **Enum-based Singleton** is the safest, cleanest, and most reliable solution.